

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ
И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ
ТЕХНОЛОГИЙ**

**Отчет по лабораторной работе №4
по курсу «Проектирование и реализация баз данных»
Тема: «Запросы на выборку и модификацию данных.
Представления. Работа с индексами»**

Выполнил:
Блинова Полина Вячеславовна К3239

Проверил:
Говорова Марина Михайловна

Санкт-Петербург
2026 г.

Содержание

<i>1 Цель работы и практическое задание</i>	<i>3</i>
<i>2 Схема базы данных (ЛР 3).....</i>	<i>4</i>
<i>3 Выполнение задания</i>	<i>5</i>
3.1 Запросы к БД (задание 2 индивидуального задания ЛР 2)	5
3.2 Представления (задание 3 индивидуального задания ЛР 2).....	16
3.3 Запросы на модификацию данных (INSERT, UPDATE, DELETE) с подзапросами.....	20
3.4 Изучение графического представления запросов и истории запросов ..	26
3.5 Создание индексов и сравнение производительности	27
<i>Вывод</i>	<i>27</i>

1 Цель работы и практическое задание

Цель работы - овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

- Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
- Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
- Изучить графическое представление запросов и просмотреть историю запросов.
- Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Наименование БД – “Courses”

3 Выполнение задания

3.1 Запросы к БД (задание 2 индивидуального задания ЛР 2)

База данных: Courses, схема: courses_schema

Запрос 1. Вывести все номера групп и программы, где количество слушателей меньше 10

Формулировка:

Получить список групп, в которых обучается менее 10 студентов, с указанием номера группы и названия образовательной программы.

```
SELECT
    g.group_number AS "Номер группы",
    ep.name AS "Программа",
    COUNT(ge.individual_id) AS "Количество слушателей"
FROM courses_schema.group_db g
JOIN courses_schema.education_program ep ON g.program_id = ep.program_id
LEFT JOIN courses_schema.group_enrollment ge ON g.group_id = ge.group_id
GROUP BY g.group_id, g.group_number, ep.name
HAVING COUNT(ge.individual_id) < 10
ORDER BY "Количество слушателей";
```

Скриншот:

```
1 SELECT
2     g.group_number AS "Номер группы",
3     ep.name AS "Программа",
4     COUNT(ge.individual_id) AS "Количество слушателей"
5 FROM courses_schema.group_db g
6 JOIN courses_schema.education_program ep ON g.program_id = ep.program_id
7 LEFT JOIN courses_schema.group_enrollment ge ON g.group_id = ge.group_id
8 GROUP BY g.group_id, g.group_number, ep.name
9 HAVING COUNT(ge.individual_id) < 10
10 ORDER BY "Количество слушателей";
11
```

Data Output Messages Notifications

	Номер группы integer	Программа text	Количество слушателей bigint
1	2	Веб-разработка (ИТМ...	1
2	1	Data Science (ИТМО)	2
3	1	DevOps инжиниринг	2
4	2	Data Science (ИТМО)	3
5	1	Кибербезопасность	4

Total rows: 5 Query complete 00:00:02.577

Запрос 2. Вывести список преподавателей с указанием количества программ, где они преподавали за истекший учебный год

Формулировка:

Для каждого преподавателя подсчитать, сколько различных образовательных программ он вёл за прошлый учебный.

SELECT

```

    courses_schema.individual.full_name AS "Преподаватель",

    COUNT(DISTINCT courses_schema.education_program.program_id) AS
"Количество программ"

FROM

    courses_schema.teacher,

    courses_schema.individual,

    courses_schema.lesson,

    courses_schema.subject,

    courses_schema.program_subject,

    courses_schema.education_program

WHERE

    courses_schema.teacher.individual_id =
courses_schema.individual.individual_id

    AND courses_schema.teacher.teacher_id = courses_schema.lesson.teacher_id

    AND courses_schema.lesson.subject_id = courses_schema.subject.subject_id

    AND courses_schema.subject.subject_id =
courses_schema.program_subject.subject_id

    AND courses_schema.program_subject.program_id =
courses_schema.education_program.program_id

    AND courses_schema.lesson.lesson_datetime >= '2025-09-01'

    AND courses_schema.lesson.lesson_datetime < '2026-09-01'

GROUP BY

    courses_schema.teacher.teacher_id,

    courses_schema.individual.full_name

ORDER BY

    "Количество программ" DESC;

```

Скриншот:

Query
Query History
AI Assistant

```

1  SELECT
2      courses_schema.individual.full_name AS "Преподаватель",
3      COUNT(DISTINCT courses_schema.education_program.program_id) AS "Количество программ"
4  FROM
5      courses_schema.teacher,
6      courses_schema.individual,
7      courses_schema.lesson,
8      courses_schema.subject,
9      courses_schema.program_subject,
10     courses_schema.education_program
11 WHERE
12     courses_schema.teacher.individual_id = courses_schema.individual.individual_id
13     AND courses_schema.teacher.teacher_id = courses_schema.lesson.teacher_id
14     AND courses_schema.lesson.subject_id = courses_schema.subject.subject_id
15     AND courses_schema.subject.subject_id = courses_schema.program_subject.subject_id
16     AND courses_schema.program_subject.program_id = courses_schema.education_program.program_id
17     AND courses_schema.lesson.lesson_datetime >= '2025-09-01'
18     AND courses_schema.lesson.lesson_datetime < '2026-09-01'
19 GROUP BY
20     courses_schema.teacher.teacher_id,
21     courses_schema.individual.full_name
22 ORDER BY
23     "Количество программ" DESC;
24

```

Data Output
Messages
Notifications

Showing rows: 1 to 4

	Преподаватель text	Количество программ bigint
1	Кузнецова Алина Сергеевна	2
2	Смирнов Никита Олегович	1
3	Соколов Дмитрий Алексеевич	1
4	Михайлова Анна Владимиров...	1

Total rows: 4
Query complete 00:00:02.440

Запрос 3. Вывести список преподавателей, которые не проводят занятия на третьей паре ни в один из дней недели

Формулировка:

Найти преподавателей, у которых нет занятий с pair_number = 3 ни в один день недели.

```

SELECT DISTINCT
    i.full_name AS "Преподаватель",
    t.employment_status AS "Статус"
FROM courses_schema.teacher t
JOIN courses_schema.individual i ON t.individual_id = i.individual_id
WHERE t.teacher_id NOT IN (
    SELECT DISTINCT l.teacher_id
    FROM courses_schema.lesson l
    WHERE l.pair_number = 3
)
ORDER BY i.full_name;

```

Скриншот:

Query

Query History

AI Assistant

1

2

3

4

5

6

7

8

9

10

11

SELECT DISTINCT

i.full_name AS "Преподаватель"

FROM courses_schema.teacher t

JOIN courses_schema.individual i ON t.individual_id = i.individual_id

WHERE t.teacher_id NOT IN (

SELECT DISTINCT l.teacher_id

FROM courses_schema.lesson l

WHERE l.pair_number = 3

)

ORDER BY i.full_name;

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Show

	Преподаватель	
	text	
1	Соколов Дмитрий Алексеевич	

Total rows: 1

Query complete 00:00:01.709

Запрос 4. Вывести список свободных лекционных аудиторий на ближайший понедельник

Формулировка:

Найти все аудитории с типом 'Лекционная', у которых нет занятий в ближайший понедельник.

```
SELECT
    courses_schema.room.room_number AS "Аудитория",
    courses_schema.room.capacity AS "Вместимость"
FROM
    courses_schema.room
WHERE
    courses_schema.room.room_type = 'Лекционная'
    AND NOT EXISTS (
        SELECT 1
        FROM courses_schema.lesson
        WHERE courses_schema.lesson.room_id = courses_schema.room.room_id
            AND DATE(courses_schema.lesson.lesson_datetime) = CURRENT_DATE + (8 -
EXTRACT(DOW FROM CURRENT_DATE)::integer) % 7
    )
ORDER BY
    courses_schema.room.room_number;
```

Скриншот:

Courses/postgres@PostgreSQL 18

Query Query History AI Assistant

```

1  SELECT
2      courses_schema.room.room_number AS "Аудитория",
3      courses_schema.room.capacity AS "Вместимость"
4  FROM
5      courses_schema.room
6  WHERE
7      courses_schema.room.room_type = 'Лекционная'
8      AND NOT EXISTS (
9          SELECT 1
10         FROM courses_schema.lesson
11         WHERE courses_schema.lesson.room_id = courses_schema.room.room_id
12               AND DATE(courses_schema.lesson.lesson_datetime) = CURRENT_DATE + (8 - EXTRACT(DOW FROM CURRENT_DATE)::integer) % 7
13     )
14  ORDER BY
15      courses_schema.room.room_number;

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	Аудитория text	Вместимость integer
1	A-101	80

Total rows: 1 Query complete 00:00:03.927 LF Ln 15,

Запрос 5. Вычислить общее количество обучающихся по каждой программе за последний год

Формулировка:

Для каждой программы подсчитать количество студентов, которые были зачислены на группы этой программы за последние 365 дней.

```

SELECT
    ep.name AS "Программа",
    COUNT(DISTINCT ge.individual_id) AS "Количество обучающихся"
FROM courses_schema.education_program ep
JOIN courses_schema.group_db g ON ep.program_id = g.program_id
JOIN courses_schema.group_enrollment ge ON g.group_id = ge.group_id
WHERE ge.academic_year = '2025/2026'
GROUP BY ep.program_id, ep.name
ORDER BY "Количество обучающихся" DESC;

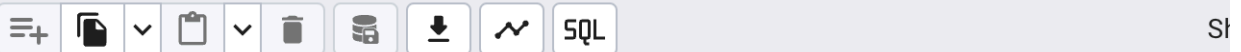
```

Скриншот:

```

1  SELECT
2      ep.name AS "Программа",
3      COUNT(DISTINCT ge.individual_id) AS "Количество обучающихся"
4  FROM courses_schema.education_program ep
5  JOIN courses_schema.group_db g ON ep.program_id = g.program_id
6  JOIN courses_schema.group_enrollment ge ON g.group_id = ge.group_id
7  WHERE ge.academic_year = '2025/2026'
8  GROUP BY ep.program_id, ep.name
9  ORDER BY "Количество обучающихся" DESC;
10

```



	Программа text	Количество обучающихся bigint
1	Кибербезопасность	4
2	Data Science (ИТМО)	2
3	DevOps инжиниринг	2
4	Веб-разработка (ИТМ...	1

Запрос 6. Вычислить среднюю загруженность компьютерных классов в неделю за последний месяц (в часах)

Формулировка:

За последние 30 дней рассчитать среднее количество часов занятий в неделю для аудиторий типа 'Компьютерный класс'.

```
SELECT
    room_number AS "Аудитория",
    ROUND(AVG(weekly_hours), 2) AS "Средняя загруженность"
FROM (
    SELECT
        r.room_id,
        r.room_number,
        DATE_TRUNC('week', l.lesson_datetime) AS week_start,
        COUNT(*) * 1.5 AS weekly_hours
    FROM
        courses_schema.lesson l
        JOIN courses_schema.room r ON l.room_id = r.room_id
    WHERE
        r.room_type = 'Компьютерный класс'
        AND l.lesson_datetime >= CURRENT_DATE - 30
    GROUP BY
        r.room_id,
        r.room_number,
        DATE_TRUNC('week', l.lesson_datetime)
) AS weekly
GROUP BY
    room_id,
    room_number
ORDER BY
    "Средняя загруженность" DESC;
```

Скриншот:

Query

Query History

AI Assistant

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

```
SELECT
    room_number AS "Аудитория",
    ROUND(AVG(weekly_hours), 2) AS "Средняя загруженность"
FROM (
    SELECT
        r.room_id,
        r.room_number,
        DATE_TRUNC('week', l.lesson_datetime) AS week_start,
        COUNT(*) * 1.5 AS weekly_hours
    FROM
        courses_schema.lesson l
    JOIN courses_schema.room r ON l.room_id = r.room_id
    WHERE
        r.room_type = 'Компьютерный класс'
        AND l.lesson_datetime >= CURRENT_DATE - 30
    GROUP BY
        r.room_id,
        r.room_number,
        DATE_TRUNC('week', l.lesson_datetime)
) AS weekly
GROUP BY
    room_id,
    room_number
ORDER BY
    "Средняя загруженность" DESC;
```

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

▼

📦

▼

⬇️

▼

📶

▼

SQL

	Аудитория text 🔒	Средняя загруженность numeric 🔒
1	Б-202	2.25
2	В-404	1.50
3	Г-505	1.50

Total rows: 3

Query complete 00:00:01.717

Query Query History AI Assistant

```
1 SELECT
2     ep.name AS "Программа",
3     COUNT(cd.completion_doc_id) AS "Количество выпускников"
4 FROM courses_schema.education_program ep
5 JOIN courses_schema.completion_document cd ON ep.program_id = cd.program_id
6 WHERE cd.issue_date >= CURRENT_DATE - INTERVAL '3 years'
7 GROUP BY ep.program_id, ep.name
8 ORDER BY "Количество выпускников" DESC;
9
```

Data Output Messages Notifications



	Программа text	Количество выпускников bigint
1	Data Science (ИТМО)	5
2	Веб-разработка (ИТМ...	2

Total rows: 2 Query complete 00:00:02.244

3.2 Представления (задание 3 индивидуального задания ЛР 2)

Представление 1. Для потенциальных слушателей: перечень специальностей (программ), изучаемых на них дисциплин и количество часов

Формулировка:

Создать представление, которое показывает абитуриентам и потенциальным слушателям все образовательные программы с перечнем входящих в них дисциплин и объёмом часов по каждой дисциплине.

```
CREATE OR REPLACE VIEW courses_schema.programs_for_applicants AS
SELECT
    ep.name AS "Образовательная программа",
    ep.program_type AS "Тип программы",
    ep.total_hours AS "Общая трудоёмкость (часов)",
    ep.document_type AS "Документ об окончании",
    s.name AS "Дисциплина",
    ps.hours_in_program AS "Часов по дисциплине",
    ps.semester AS "Семестр",
    ps.control_form AS "Форма контроля"
FROM courses_schema.education_program ep
JOIN courses_schema.program_subject ps ON ep.program_id = ps.program_id
JOIN courses_schema.subject s ON ps.subject_id = s.subject_id
ORDER BY ep.name, ps.semester, s.name;
```

Скриншоты:

```
1 CREATE OR REPLACE VIEW courses_schema.programs_for_applicants AS
2 SELECT
3     ep.name AS "Образовательная программа",
4     ep.program_type AS "Тип программы",
5     ep.total_hours AS "Общая трудоёмкость",
6     ep.document_type AS "Документ об окончании",
7     s.name AS "Дисциплина",
8     ps.hours_in_program AS "Часов по дисциплине",
9     ps.semester AS "Семестр",
10    ps.control_form AS "Форма контроля"
11 FROM courses_schema.education_program ep
12 JOIN courses_schema.program_subject ps ON ep.program_id = ps.program_id
13 JOIN courses_schema.subject s ON ps.subject_id = s.subject_id
14 ORDER BY ep.name, ps.semester, s.name;
15
```

CREATE VIEW

Query returned successfully in 3 secs 744 msec.

Total rows: Query complete 00:00:03.744

```
1 SELECT * FROM courses_schema.programs_for_applicants
```

Showing rows: 1 to 9 of 1								
	Образовательная программа text	Тип программы text	Общая трудоёмкость integer	Документ об окончании text	Дисциплина text	Часов по дисциплине integer	Семестр integer	Форма кон text
1	Data Science (ИТМО)	Профессиональная переподготов...	520	Диплом	Python программирование	180	1	Экзамен
2	Data Science (ИТМО)	Профессиональная переподготов...	520	Диплом	Базы данных	120	1	Экзамен
3	Data Science (ИТМО)	Профессиональная переподготов...	520	Диплом	Машинное обучение	200	2	Экзамен
4	DevOps инжиниринг	Повышение квалификации	300	Удостоверение	CI/CD пайплайны	60	1	Зачет
5	DevOps инжиниринг	Повышение квалификации	300	Удостоверение	Контейнеризация и оркестрац...	120	2	Экзамен
6	Веб-разработка (ИТМО)	Повышение квалификации	260	Удостоверение	Веб-технологии	140	1	Зачет
7	Кибербезопасность	Профессиональная переподготов...	450	Диплом	Основы кибербезопасности	80	1	Экзамен
8	Кибербезопасность	Профессиональная переподготов...	450	Диплом	Компьютерные сети	80	2	Зачет
9	Кибербезопасность	Профессиональная переподготов...	450	Диплом	Сети и безопасность	100	2	Экзамен

Total rows: 9 Query complete 00:00:02.930

LF Ln 1, Col 53

Представление 2. Общий доход по каждой программе за последний год

Формулировка:

Создать представление, которое вычисляет общий доход (например, от стоимости обучения) по каждой образовательной программе за последний год обучения.

Доход считался, как фиксированная плата за обучения на количество студентов.

```
CREATE OR REPLACE VIEW courses_schema.program_income_last_year_simple AS
SELECT
    ep.name AS "Программа",
    ep.program_type AS "Тип программы",
    COUNT(DISTINCT ge.individual_id) AS "Количество студентов",
    ROUND(SUM(ge.tuition_fee), 2) AS "Общий доход"
FROM
    courses_schema.education_program ep
    JOIN courses_schema.group_db g ON ep.program_id = g.program_id
    JOIN courses_schema.group_enrollment ge ON g.group_id = ge.group_id
GROUP BY
    ep.program_id,
    ep.name,
    ep.program_type
ORDER BY
    "Общий доход" DESC;
```

Скриншоты:

QueryQuery HistoryAI Assistant

```
1 CREATE VIEW courses_schema.program_income_last_year_simple AS
2 SELECT
3     ep.name AS "Программа",
4     ep.program_type AS "Тип программы",
5     COUNT(DISTINCT ge.individual_id) AS "Количество студентов",
6     ROUND(SUM(ge.tuition_fee), 2) AS "Общий доход"
7 FROM
8     courses_schema.education_program ep
9 JOIN courses_schema.group_db g ON ep.program_id = g.program_id
10 JOIN courses_schema.group_enrollment ge ON g.group_id = ge.group_id
11 GROUP BY
12     ep.program_id,
13     ep.name,
14     ep.program_type
15 ORDER BY
16     "Общий доход" DESC;
```

Data OutputMessagesNotifications

CREATE VIEW

Query returned successfully in 709 msec.

Total rows: Query complete 00:00:00.709

QueryQuery HistoryAI Assistant

```
1 SELECT * FROM courses_schema.program_income_last_year_simple;
```

Data OutputMessagesNotifications

	Программа text	Тип программы text	Количество студентов bigint	Общий доход numeric
1	Data Science (ИТМО)	Профессиональная переподготов...	3	600000.00
2	Кибербезопасность	Профессиональная переподготов...	4	600000.00
3	DevOps инжиниринг	Повышение квалификации	2	180000.00
4	Веб-разработка (ИТМ...	Повышение квалификации	1	80000.00

Total rows: 4Query complete 00:00:03.543

3.3 Запросы на модификацию данных (INSERT, UPDATE, DELETE) с подзапросами

Запрос 1. INSERT. Добавление нового зачисления для студента с наибольшим ID

Формулировка:

Зачислить самого нового студента (с максимальным individual_id) в группу, где меньше всего студентов.

```
INSERT INTO courses_schema.group_enrollment (enrollment_id, individual_id,
group_id, academic_year, tuition_fee)
SELECT
    (SELECT MAX(enrollment_id) + 1 FROM courses_schema.group_enrollment),
    (SELECT MAX(individual_id) FROM courses_schema.individual),
    group_id,
    '2025/2026',
    50000
FROM (
    SELECT g.group_id
    FROM courses_schema.group_db g
    LEFT JOIN courses_schema.group_enrollment ge ON g.group_id = ge.group_id
    GROUP BY g.group_id
    ORDER BY COUNT(ge.individual_id) ASC
    LIMIT 1
) AS least_populated_group;
```

Скриншоты (с 2 проверками до, выполнением и проверкой после):

Query Query History AI Assistant

1 SELECT individual_id, full_name

2 FROM courses_schema.individual

3 ORDER BY individual_id DESC

4 LIMIT 1;

Data Output Messages Notifications

	individual_id	full_name
	[PK] bigint	text
1	21	Кузьмина Ирина Викторов...

Total rows: 1 Query complete 00:00:02.952

Query Query History AI Assistant

1 SELECT

2 g.group_id,

3 g.group_number,

4 COUNT(ge.individual_id) AS student_count

5 FROM courses_schema.group_db g

6 LEFT JOIN courses_schema.group_enrollment ge ON g.group_id = ge.group_id

7 GROUP BY g.group_id, g.group_number

8 ORDER BY student_count;

9

Data Output Messages Notifications

	group_id	group_number	student_count
	[PK] bigint	integer	bigint
1	2	2	1
2	4	1	2
3	1	1	2
4	5	2	3
5	3	1	4

Total rows: 5 Query complete 00:00:01.411

[Query](#) [Query History](#) [AI Assistant](#)

```

1 INSERT INTO courses_schema.group_enrollment (enrollment_id, individual_id, group_id, academic_year, tuition_fee)
2 SELECT
3     (SELECT MAX(enrollment_id) + 1 FROM courses_schema.group_enrollment),
4     (SELECT MAX(individual_id) FROM courses_schema.individual),
5     group_id,
6     '2025/2026',
7     50000
8 FROM (
9     SELECT g.group_id
10    FROM courses_schema.group_db g
11   LEFT JOIN courses_schema.group_enrollment ge ON g.group_id = ge.group_id
12  GROUP BY g.group_id
13  ORDER BY COUNT(ge.individual_id)
14  LIMIT 1
15 ) AS least_populated_group;

```

Data Output **Messages** Notifications

```
INSERT @ 1
```

Query returned successfully in 697 msec.

Total rows: Query complete 00:00:00.697

Query Query History AI Assistant

```
1 SELECT
2     ge.individual_id,
3     i.full_name,
4     ge.group_id,
5     g.group_number,
6     ge.academic_year
7 FROM courses_schema.group_enrollment ge
8 JOIN courses_schema.individual i ON ge.individual_id = i.individual_id
9 JOIN courses_schema.group_db g ON ge.group_id = g.group_id
10 WHERE ge.individual_id = (SELECT MAX(individual_id) FROM courses_schema.individual);
```

Data Output Messages Notifications

									SQL	Showin
	individualId bigint	full_name text	group_id bigint	group_number integer	academic_year text					
1	21	Кузьмина Ирина Викторов...	2	2	2025/2026					

Total rows: 1 Query complete 00:00:03.292

Запрос 2. UPDATE. Повышение зарплаты преподавателям, которые ведут больше 2 дисциплин

Формулировка:

Повысить зарплату преподавателям на 10% процентов, которые ведут больше 2 дисциплин.

```
UPDATE courses_schema.teacher
SET salary = salary * 1.1
WHERE teacher_id IN (
    SELECT teacher_id
    FROM courses_schema.lesson
    GROUP BY teacher_id
    HAVING COUNT(DISTINCT subject_id) > 2
);
```

Скриншоты (проверка до, выполнение и проверка после):

Query Query History AI Assistant

```
1 SELECT teacher_id, salary FROM courses_schema.teacher ORDER BY teacher_id;
```

Data Output Messages Notifications

	teacher_id [PK] bigint	salary numeric (10,2)
1	1	120000.00
2	2	90000.00
3	3	75000.00
4	4	85000.00
5	5	95000.00
6	20	50000.00

Total rows: 6 Query complete 00:00:01.281

Query Query History AI Assistant

```
1 UPDATE courses_schema.teacher
2 SET salary = salary * 1.1
3 WHERE teacher_id IN (
4     SELECT teacher_id
5     FROM courses_schema.lesson
6     GROUP BY teacher_id
7     HAVING COUNT(DISTINCT subject_id) > 2
8 );
```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 2 secs 751 msec.

Total rows: Query complete 00:00:02.751

Query Query History AI Assistant

```
1 SELECT teacher_id, salary FROM courses_schema.teacher ORDER BY teacher_id;
```

Data Output Messages Notifications

	teacher_id [PK] bigint	salary numeric (10,2)
1	1	132000.00
2	2	90000.00
3	3	75000.00
4	4	85000.00
5	5	95000.00
6	20	50000.00

Total rows: 6 Query complete 00:00:00.819

Запрос 3. DELETE. Удаление старых занятий

Формулировка:

Удалить все занятия, которые были проведены более года назад (365 дней).

```
DELETE FROM courses_schema.lesson
WHERE lesson_id IN (
    SELECT lesson_id
    FROM courses_schema.lesson
    WHERE lesson_datetime < CURRENT_DATE - 365
);
```

Скриншоты (проверка до, выполнение и проверка после):

QueryQuery HistoryAI Assistant

123

SELECT lesson_id, lesson_datetime, lesson_type
FROM courses_schema.lesson
WHERE lesson_datetime < CURRENT_DATE - 365;

QueryQuery HistoryAI Assistant

123456

DELETE FROM courses_schema.lesson
WHERE lesson_id IN (
 SELECT lesson_id
 FROM courses_schema.lesson
 WHERE lesson_datetime < CURRENT_DATE - 365
);

Data OutputMessagesNotifications

lesson_id
[PK] bigint

lesson_datetime
timestamp with time zone

lesson_type
text

1	3	2024-10-10 10:00:00+03	Лекция
2	4	2024-10-11 12:00:00+03	Лекция
3	5	2024-10-12 14:00:00+03	Лекция

Total rows: 3Query complete 00:00:01.753

Data OutputMessagesNotifications

DELETE 3

Query returned successfully in 1 secs 291 msec.

Total rows:Query complete 00:00:01.291

Query

Query History

AI Assistant

1

2

3

SELECT COUNT(*) AS old_lessons_remaining

FROM courses_schema.lesson

WHERE lesson_datetime < CURRENT_DATE - 365;

Data Output

Messages

Notifications

SQL

old_lessons_remaining

bigint

1

0

Total rows: 1

Query complete 00:00:01.543

3.4 Изучение графического представления запросов и истории запросов

Explain

QueryQuery HistoryAI Assistant

1SELECT COUNT(*) AS old_lessons_remaining

2FROM courses_schema.lesson

3WHERE lesson_datetime < CURRENT_DATE - 365;

Data OutputMessagesExplain XNotifications

GraphicalAnalysisStatistics

lesson

Aggregate

Total rows: 1

Query complete 00:00:01.377

История запросов:

QueryQuery HistoryAI Assistant

Show queries generated internally by pgAdmin?

Remove

Remove All

Today - 4/13/2026

E

SELECT * FROM courses_schema.individual WHERE full_name LIKE 'Ивано...

12:26:03

▶

SELECT * FROM courses_schema.individual WHERE full_name LIKE 'Иванов...

12:25:53

E

SELECT COUNT(*) AS old_lessons_remaining FROM courses_schema.lesson...

12:24:52

▶

SELECT COUNT(*) AS old_lessons_remaining FROM courses_schema.lesson ...

12:23:26

▶

DELETE FROM courses_schema.lesson WHERE lesson_id IN (SELECT lesson...

12:23:00

▶

SELECT lesson_id, lesson_datetime, lesson_type FROM courses_schema.l...

12:22:33

▶

SELECT ge.individual_id, i.full_name, ge.group_id, g.group_number, g...

12:19:59

▶

INSERT INTO courses_schema.group_enrollment (enrollment_id, individu...

12:18:26

▶

DELETE FROM courses_schema.group_enrollment WHERE enrollment_id = (S...

4/13/2026 12:26:03 PM

1

1 secs 391 msec

Date

Rows affected

Duration

Copy

Copy to Query Editor

SELECT * FROM courses_schema.individual WHERE full_name LIKE 'Иванов%';

Messages

Successfully run. Total query runtime: 1 secs 391 msec. 1 rows affected.

Data OutputMessagesNotifications

Recorded time

Event

Process ID

Payload

3.5 Создание индексов и сравнение производительности

Простой индекс:

Создание индекса

[Query](#) [Query History](#) [AI Assistant](#)

1

2

```
CREATE INDEX idx_individual_name
ON courses_schema.individual(full_name);
```

Data Output

[Messages](#)

[Explain](#)

[×](#)

[Notifications](#)

CREATE INDEX

Query returned successfully in 4 secs 412 msec.

Total rows:

Query complete 00:00:04.597

Тест без индекса (время 06.137 с):

QueryQuery HistoryAI Assistant

1SELECT * FROM courses_schema.individual

2WHERE full_name = 'Кузьмина Ирина Викторовна';

Data OutputMessagesExplain XNotifications

GraphicalAnalysisStatistics

#	Node	Rows		Loops
		Actual		
1.	Seq Scan on individual as individual (rows=1 loops=1) Filter: (full_name = 'Кузьмина Ирина Викторовна'::text) Rows Removed by Filter: 120		1	1

Total rows: 1

Query complete 00:00:06.137

LF Ln 2, Col 45

Затем специально отключила seq scan и проверила. Время 01.089 с:

QueryQuery HistoryAI Assistant

1SET enable_seqscan = OFF;

2

3EXPLAIN ANALYZE

4SELECT * FROM courses_schema.individual

5WHERE full_name = 'Кузьмина Ирина Викторовна';

6

7SET enable_seqscan = ON;

Data OutputMessagesExplain XNotifications

Total rows:

Query complete 00:00:01.089

Составной индекс:

Тест без индекса (время 0.744 с):

QueryQuery HistoryAI Assistant

1234

SELECT *
FROM courses_schema.group_enrollment
WHERE group_id = 1
AND academic_year = '2025/2026';

Data OutputMessagesExplain XNotifications

GraphicalAnalysisStatistics

#	Node
1.	→ Seq Scan on group_enrollment as group_enrollment (rows=2 loops=1) Filter: (((group_id = 1) AND (academic_year = '2025/2026'::text)) Rows Removed by Filter: 11

Total rows: 1

Query complete 00:00:00.744

Создание индекса:

QueryQuery HistoryAI Assistant

12

CREATE INDEX idx_group_enrollment_group_year
ON courses_schema.group_enrollment(group_id, academic_year);

Data OutputMessagesExplain XNotifications

CREATE INDEX

Query returned successfully in 3 secs 492 msec.

Total rows:

Query complete 00:00:03.681

Затем специально отключила seq scan и проверила. Время 0.460 с:

[Query](#) [Query History](#) [AI Assistant](#)

```
1 SET enable_seqscan = OFF;  
2  
3 EXPLAIN ANALYZE  
4 SELECT *  
5 FROM courses_schema.group_enrollment  
6 WHERE group_id = 1  
7     AND academic_year = '2025/2026';  
8  
9 SET enable_seqscan = ON;
```

[Data Output](#) [Messages](#) [Explain](#) [×](#) [Notifications](#)

SET

Query returned successfully in 460 msec.

Total rows:	Query complete 00:00:00.460
-------------	-----------------------------

Вывод

В ходе выполнения лабораторной работы №4 были созданы 7 запросов на выборку данных для БД «Курсы», а также 2 представления для потенциальных слушателей и расчёта дохода. Составлены запросы на модификацию данных с использованием подзапросов. Изучен графический план выполнения запросов (Explain) и история запросов в pgAdmin 4. Созданы простой и составной индексы. Индексы являются эффективным инструментом оптимизации производительности запросов, особенно для полей, часто используемых в условиях WHERE, JOIN и ORDER BY.